

VITESSE

Arquitectura Operativa v11

WhatsApp Messaging Platform on GCP · Abril 2026

Modelo actualizado: vitesse_msg_id como raíz de conversación, /active como capa operativa, Pub/Sub como backbone transaccional del pipeline, Firestore como proyección viva, DLR colapsados para frontend y Cloud Storage como event log.

Objetivo de esta versión. Cerrar la semántica de identificadores, simplificar el hot path del frontend y eliminar dependencias innecesarias de Cloud SQL en el flujo operativo.

1. Resumen ejecutivo

La arquitectura propuesta para Vitesse se apoya en tres capas complementarias: Cloud Storage como registro inmutable de eventos, Pub/Sub como backbone transaccional de transporte, buffering y reintentos, y Firestore como índice/proyección viva para resolución de conversaciones, frontend y correlación por gsld/wamid.

La decisión central de esta versión es semántica: en Vitesse, vitesse_msg_id deja de ser leído como "id de mensaje individual" y pasa a ser el identificador raíz de la conversación. vitesse_msg_id nace en dos casos canónicos: (1) en el outbound inicial originado desde una campaña o carga masiva, cuando se parte el archivo por destinatario; y (2) en un inbound sin correlación previa, sin conversación activa y sin campaña reutilizable para ese número. En ese segundo caso se crea además una campaña comodín mensual. Cada envío outbound genera sus propios gsld y wamid; cada inbound trae su propio wamid; pero todos esos identificadores externos resuelven al mismo vitesse_msg_id mientras pertenezcan al mismo hilo. En paralelo, todo evento operativo — inbound, outbound, status, compresión y limpieza — se desacopla mediante Pub/Sub antes de ser procesado por la siguiente etapa.

Conclusión operativa. Storage resuelve durabilidad y costo; Firestore resuelve lookups y UX; Cloud SQL queda fuera del hot path y se reserva para proyecciones opcionales de reporting, billing o exportación.

Principios rectores

Principio	Decisión	Resultado
-----------	----------	-----------

Conversación como entidad raíz	vitesse_msg_id = raíz de conversación	Todos los gsld/wamid se correlacionan contra un mismo hilo
Event log barato	/active + /archived + /data en Cloud Storage	Replay, compresión, auditoría y analítica sin saturar la BD
Lookup rápido	Firestore para índices y proyección viva	Frontend y correlación sin listar objetos en Storage
Frontend por push	Firestore + push desde backend	No polling, menor latencia visible
Campaign siempre presente	campaign_id real o sintético mensual	Orden operativo, dashboard y trazabilidad comercial

Regla de diseño: Parser/Receptor persisten el mínimo necesario y publican; Processor/Worker consumen desde Pub/Sub y escriben sus efectos en Storage y/o Firestore. Ni el frontend ni los workers deben depender de invocaciones directas entre componentes de negocio.

Todo absolutamente todo pasa por Pub/Sub. Cada evento operativo o comando interno — carga de campaña, envío outbound, webhook inbound, DLR/status, actualización de proyección, compresión, limpieza y maintenance — debe publicarse en un topic antes de ser procesado por la siguiente etapa. Pub/Sub es la capa única de desacoplamiento, buffering, reintento y DLQ del sistema.

Pub/Sub como backbone transaccional

2. Modelo canónico de identificadores

El modelo actualizado trabaja con un identificador interno raíz y dos identificadores externos principales. La clave es no mezclar ámbitos: vitesse_msg_id agrupa la conversación; gsld agrupa un outbound específico en Gupshup; wamid identifica mensajes concretos de WhatsApp/Meta.

Identificador	Generado por	Qué representa	Uso
vitesse_msg_id	Vitesse	Raíz de conversación	Nace en el outbound inicial o en un inbound huérfano sin match; agrupa el hilo, frontend y campaign
gsld	Gupshup	Outbound específico y sus DLR	enqueued/sent/delivered/read/failed del mismo envío
wamid	Meta/WhatsApp	Mensaje concreto de WhatsApp	Inbound del usuario o message id real del outbound
campaign_id	Vitesse	Origen comercial u operativo	Campaña real o sintética (organic_YYYY_MM)

Regla canónica. Todo gsld debe resolver a un vitesse_msg_id. Todo wamid debe resolver a un vitesse_msg_id. Un mismo vitesse_msg_id puede tener múltiples gsld y múltiples wamid a lo largo del hilo.

Mapa de conversación con cinco interacciones

vitesse_msg_id = vm_20260412_0001

```
|
|—— outbound #1 -> gsld=gs_aaa -> wamid=wa_out_111 -> enqueued/sent/delivered/read
|—— inbound #1 -> wamid=wa_in_222 -> "Hola"
|—— outbound #2 -> gsld=gs_bbb -> wamid=wa_out_333 -> enqueued/sent/delivered/read
|—— inbound #2 -> wamid=wa_in_444 -> "Quiero saber el estado de mi pedido"
|—— outbound #3 -> gsld=gs_ccc -> wamid=wa_out_555 -> enqueued/sent/delivered/read
|—— inbound #3 -> wamid=wa_in_666 -> "Es el 458921"
|—— outbound #4 -> gsld=gs_ddd -> wamid=wa_out_777 -> enqueued/sent/delivered/read
|—— inbound #4 -> wamid=wa_in_888 -> "Perfecto, gracias"
|—— outbound #5 -> gsld=gs_eee -> wamid=wa_out_999 -> enqueued/sent/delivered/read
```

Nota: si un inbound llega sin context y sin active_conv, nace un nuevo vitesse_msg_id y se asigna campaign_id sintético mensual.

3. Capas de almacenamiento y responsabilidades

La arquitectura se simplifica cuando cada servicio resuelve una sola responsabilidad. Storage no intenta responder consultas de frontend; Firestore no intenta reemplazar el histórico; y Pub/Sub se convierte en la capa obligatoria de desacoplamiento entre productores y consumidores. Ningún CRF o job de negocio debería invocar directamente a otro en el hot path.

Capa	Tecnología	Responsabilidad	Uso principal
/active	Cloud Storage	Estado operativo de eventos	todo / done / error
Firestore	Document DB serverless	Índices de correlación y vista viva	frontend, lookup, resolución de contexto
/archived	Cloud Storage	Audit trail histórico comprimido	jsonl.gz, replay, cumplimiento
/data	Cloud Storage + Parquet	Explotación analítica	BI, billing, joins por lote
Cloud SQL (opcional)	Relacional administrado	Proyecciones administrativas	reporting puntual, exportes o billing si hace falta

Por qué no Cloud SQL en el hot path. El patrón dominante del sistema no es SQL complejo sino lookup por clave y actualización corta de estado. Firestore encaja mejor para gsld->vitesse_msg_id, wamid->vitesse_msg_id y conversación activa, mientras Storage absorbe volumen barato y sin presión de conexiones.

Convención de rutas

```
/active/todo/outbound/{app}/{date}/{hour}/{campaign_id}/{vitesse_msg_id}/{event_ts}_{local_ref}.json
/active/done/outbound/{app}/{date}/{hour}/{campaign_id}/{vitesse_msg_id}/{event_ts}_{gsld}.json
/active/done/inbound/{app}/{date}/{hour}/{campaign_id}/{vitesse_msg_id}/{event_ts}_{wamid}.json
/active/done/status/{app}/{date}/{hour}/{campaign_id}/{vitesse_msg_id}/{event_ts}_{gsld}_{status}.json
/media/{app}/{YYYY}/{MM}/{hash}.{ext}
/archived/{app}/{date}.jsonl.gz
/data/{app}/{date}.parquet
/campaign/{app}/{campaign_id}/... (índice derivado, no hot path del frontend)
```

Nota: local_ref solo existe antes de que el proveedor devuelva gsld. Puede derivarse de la fila de carga, un nonce del parser o un identificador local de intento. Una vez resuelto el envío, el archivo pasa a /active/done/..._{gsld}.json. Si el CRF Sender falla antes del provider ack y el DLQ agota reintentos, el objeto no se mueve a /active/done/: se mueve a /active/error/outbound_unresolved/..._{event_ts}_{local_ref}.json y conserva el mismo local_ref para futuros replays.

4. Modelo recomendado en Firestore

Firestore no guarda el archivo completo como fuente de verdad; guarda una proyección viva y referencias mínimas al archivo persistido en Storage. Esto permite reconstruir la conversación para frontend sin listar objetos, pero conserva el histórico completo fuera de la base documental. Firestore también puede guardar colecciones auxiliares de control, como opt_out_list, cuando la plataforma necesite proteger de forma nativa la salud del canal frente a futuros broadcasts.

Regla de materialización en Firestore: recent_events guarda mensajes visibles del hilo. Los cambios de estado DLR se consolidan sobre el mismo documento del mensaje outbound mediante last_status, delivered_at, read_at y campos equivalentes. La persistencia completa de cada transición técnica se conserva exclusivamente en Storage.

Colección / documento	Clave	Contenido mínimo
conversations/{vitesse_msg_id}	vitesse_msg_id	app, phone, campaign_id, active, last_event_ts, last_status, last_preview
conversations/{vitesse_msg_id}/recent_events/{doc}	doc aleatorio	event_ts, direction, gsld, wamid, last_status, text_preview, storage_path; TTL 30 días + cap 100 mensajes visibles por

		conversación. Los DLR (enqueued/sent/delivered/read/failed) no crean docs separados: actualizan el mensaje visible.
gsid_map/{gsid}	gsid	vitesse_msg_id, last_status, last_event_ts, delivered_at, read_at; proyección colapsada del outbound y sus DLR
wamid_map/{wamid}	wamid	vitesse_msg_id, direction, last_event_ts
active_conv/{app_phone_hash}	hash(app+phone)	vitesse_msg_id activo, updated_at, expires_at (72h por defecto, configurable por tenant)
opt_out_list/{app_phone_hash}	hash(app+phone)	opted_out=true, source=keyword manual downstream, keyword, updated_at

Diseño recomendado. Usar IDs de documento no secuenciales y limitar índices a las consultas reales. event_ts se guarda como campo ordenable; el storage_path permite abrir el JSON operativo cuando se necesite auditoría o replay.

5. Resolución de inbound: context primero, fallback después

Un inbound no trae vitesse_msg_id de forma nativa. La correlación se resuelve primero por contexto del mensaje respondido y solo en ausencia de contexto se usa la conversación activa por teléfono.

Regla operativa de nacimiento. El primer outbound de campaña crea siempre un nouveau vitesse_msg_id sin consultar active_conv. En inbound, context.gsid y context.id tienen prioridad. Solo si no existe contexto ni conversación activa por (app, phone), Vitesse crea un nouveau vitesse_msg_id y asigna una campaña comodín mensual, por ejemplo organic_2026_04.

Creación concurrente de inbound huérfano. Si dos mensajes inbound de un mismo (app, phone) llegan casi simultáneamente sin context y sin conversación activa visible, la creación del nouveau vitesse_msg_id no debe ejecutarse fuera de control. El worker debe usar una transacción sobre active_conv/{app_phone_hash}: si el documento no existe, crea vitesse_msg_id + campaign_id comodín y publica el mapping; si ya existe porque otra instancia ganó por milisegundos, reutiliza ese vitesse_msg_id existente.

Opt-out nativo. El CRF Processor (o un worker derivado suscrito a inbound-events) debe interceptar palabras clave de opt-out como STOP, BAJA, CANCELAR y equivalentes por tenant/idioma. Ese evento no solo se enruta a la conversación vigente: también debe persistir un flag en opt_out_list/{app_phone_hash} para impedir futuros broadcasts hacia ese número mientras el estado siga activo.

Prioridad	Señal	Lookup	Resultado
1	context.gsId	gsid_map/{gsId}	Correlación exacta al vitesse_msg_id del outbound
2	context.id	wamid_map/{wamid}	Correlación exacta si solo viene el ID de WhatsApp
3	(app, phone) activo	active_conv/{app_phone_hash}	Fallback conversacional
4	Sin match	crear nuevo vitesse_msg_id	Nueva conversación + campaña comodín mensual

```

if event == "outbound_initial":
    vm = create_vitesse_msg_id()
    campaign_id = real_campaign_id
elif inbound.context.gsId:
    vm = gsid_map[context.gsId]
elif inbound.context.id:
    vm = wamid_map[context.id]
elif active_conv[app_phone_hash]:
    vm = active_conv[app_phone_hash]
else:
    vm = create_vitesse_msg_id()
    campaign_id = "organic_YYYY_MM"

```

6. Frontend, campañas e índices derivados

El frontend no debe reconstruir un chat listando refs en Storage. La lectura viva sale de Firestore: conversación, últimos eventos y estado visible. Storage queda como registro canónico y profundo. Pub/Sub intermedia entre Parser/Receptor y Processor/Projector; el frontend nunca consume Storage ni workers directamente. /campaign sigue vigente como índice derivado útil para dashboard, navegación y exportación.

Componente	Sigue vigente	Rol
/campaign	Sí	Índice derivado para dashboard, tracing comercial y exportación
/conv	Opcional	Solo si luego se necesita navegación/auditoría directa desde Storage
Firestore recent_events	Sí	Fuente viva del frontend sin leer Storage

Campaign siempre presente. Todo mensaje debe tener campaign_id. Si no nace de una campaña real, se asigna una campaña sintética mensual, por ejemplo organic_2026_04 o unknown_2026_04. Esto aplica especialmente al inbound sin contexto ni conversación activa, donde también nace un nuevo vitesse_msg_id.

7. Agregación, compresión y estado visible

La capa /active conserva la ventana operativa y no se usa como base de consultas del frontend. Los jobs de agregación y compresión trabajan sobre esa capa para producir trazabilidad histórica y datasets analíticos. Cloud Scheduler no dispara lógica de negocio directamente: publica comandos a Pub/Sub y los workers/CRFs correspondientes consumen esos mensajes para Aggregator, Compressor y Cleanup.

Proceso	Entrada	Salida	Observación
Aggregator	/active/done + /active/error	resúmenes por campaña o app	No es hot path del frontend
Compressor D-30	/active/done/{D-30}	/archived/*.jsonl.gz + /data/*.parquet	Elimina JSONs operativos compactados
Billing/BI	/data/*.parquet	tablas o reportes externos	Puede usar Cloud SQL o BigQuery downstream
Purge recent_events	Firestore recent_events > D-30 o >100 eventos	conserva solo ventana viva y elimina proyección sobrante	Companion job del Compressor; purga >D-30 y aplica cap por conversación. recent_events guarda mensajes visibles, no cada transición DLR

8. Recomendación de implementación

La forma más segura de migrar es cambiar primero la semántica del modelo; después fijar Pub/Sub como backbone transversal (topics, DLQ y comandos de jobs); luego incorporar Firestore para correlación; y por último mover el frontend a la proyección viva. No conviene intentar rediseñar todo en una sola iteración.

Fase	Cambio	Resultado esperado
Fase 1	Declarar vitesse_msg_id como raíz de conversación y renombrar /state -> /active	Semántica consistente del modelo
Fase 2	Fijar Pub/Sub como backbone transversal + agregar Firestore	Definir topics, DLQ y contrato de mensajes; habilitar gsid_map, wamid_map, active_conv y recent_events

Fase 3	Mantener /campaign como índice derivado y eliminar /conv si no aporta valor	Menor complejidad operativa
Fase 4	Ajustar agregación, compresión y exportes downstream	Histórico y analítica sin tocar el hot path

9. Observaciones operativas de implementación

- **DLR fuera de orden (out-of-order):** los webhooks de sent, delivered y read pueden llegar con milisegundos de diferencia y fuera de orden. El código que actualiza Firestore no debe sobrescribir last_status de forma ciega. Debe aplicar una jerarquía de estados o actualización condicional para impedir downgrades visuales (por ejemplo: nunca pasar de delivered a sent ni de read a delivered).
- **Contención temporal sobre el mismo documento:** los DLR de un mismo gsld pueden concentrarse en pocos segundos. Para reducir contención, gsld_map/{gsld} recibe la proyección técnica consolidada y recent_events solo se actualiza cuando cambia el estado visible del mensaje. Deben usarse escrituras idempotentes con merge y reintentos automáticos del SDK; si se requiere lógica leer-modificar-escribir, usar transacción.
- **TTL 30d + cap 100 mensajes visibles:** el TTL temporal y el cap por cantidad son responsabilidades distintas. El TTL de 30 días resuelve envejecimiento temporal; el cap de 100 no es nativo y debe ejecutarse en un job asíncrono compañero del Compressor, nunca en el hot path de webhooks inbound/outbound. Ese job recorta recent_events por conversación manteniendo solo mensajes visibles recientes.
- **Costo y materialización en Firestore:** recent_events no replica cada transición DLR como documento independiente. Los estados de delivery se colapsan en el mismo mensaje visible mediante last_status, delivered_at, read_at y campos equivalentes. Esta regla mantiene razonable el cap de 100 y evita inflar el costo por conversación o por campaña.
- **Reads del frontend en Firestore:** las escrituras son baratas, pero los onSnapshot sobre listas abiertas pueden disparar millones de lecturas si muchos agentes mantienen dashboards vivos durante horas. El frontend debe usar paginación estricta, scopes por agente/cola y listeners solo sobre los chats visibles o asignados en pantalla en ese momento; nunca queries abiertas de todos los chats activos.

10. Topología de Pub/Sub y contrato de mensajes

Pub/Sub se define aquí como spec operativa y no solo como principio general. Cada productor publica a un topic concreto, cada consumidor procesa desde ese topic o su DLQ correspondiente y los atributos del mensaje permiten routing, correlación y observabilidad sin obligar al consumer a abrir el archivo en Storage para decisiones básicas.

Topic	Produce	Consume	Responsabilidad	DLQ
outbound-commands	CRF Parser / Campaign Loader	CRF Sender	Comandos de envío outbound por destinatario. El API/Parser genera vitesse_msg_id de forma síncrona y luego publica el comando.	deadletter-outbound
inbound-events	CRF Receptor	CRF Processor	Mensajes inbound del usuario; resolución por context o fallback antes de proyectar conversación.	deadletter-inbound
status-events	CRF Receptor DLR	CRF Processor DLR	DLR/status: enqueued, sent, delivered, read, failed; actualización técnica y no-downgrade visual.	deadletter-status
projection-updates	CRF Processor	CRF Projector Firestore	Proyección viva en Firestore; opcional si Processor escribe Firestore directamente.	deadletter-projection
maintenance-commands	Cloud Scheduler	Aggregator / Compressor / Cleanup Workers	Comandos internos de agregación, compresión D-30, purge de recent_events y tareas de maintenance.	deadletter-maintenance

media-commands	CRF Receptor / CRF Processor	CRF Media Handler	Descarga/normaliza multimedia; guarda el asset en /media/{app}/{YYYY}/{MM}/{hash}.ext y publica solo metadata / storage_path downstream.	deadletter-media
----------------	------------------------------	-------------------	--	------------------

Regla de topología: si projection-updates no se usa como etapa separada, el CRF Processor puede escribir Firestore directamente; aun así inbound, outbound, status y maintenance deben entrar y salir por Pub/Sub.

Contrato mínimo del mensaje Pub/Sub

Los atributos de Pub/Sub transportan los campos de alta utilidad para routing, correlación y observabilidad. El payload puede limitarse a un envelope ligero porque el raw event completo y el JSON operativo viven en Storage y se referencian por storage_path.

Flujo	Atributos recomendados	Payload / body
outbound-commands	event_type, vitesse_msg_id, campaign_id, client_id/app, phone o phone_hash, storage_path, local_ref, event_ts	Envelope ligero con tipo de mensaje y puntero al JSON operativo. El API/Parser devuelve vitesse_msg_id + local_ref al caller antes de publicar.
inbound-events	event_type=inbound_message, wamid, client_id/app, campaign_id, vitesse_msg_id (si ya fue resuelto), storage_path, event_ts	Envelope ligero con tipo inbound y puntero al raw webhook. Si detecta multimedia, dispara media-commands con metadata y no con binario.
status-events	event_type=status_event, gsld, status, client_id/app, campaign_id, vitesse_msg_id (si ya fue resuelto), storage_path, event_ts	Envelope ligero con estado y puntero al raw webhook. gsld y status deben viajar como atributos para lookup sin abrir el archivo.
projection-updates	event_type=projection_update, vitesse_msg_id, client_id/app, campaign_id, storage_path, entity_target=firestore	Mensaje derivado con intención de proyección; no necesita raw completo si el Projector puede leer Storage.
maintenance-commands	event_type=maintenance_command, command_type=aggregate compress cleanup, target_date, client_id/app opcional	Comando interno con fecha objetivo y parámetros del job.

media-commands	event_type=media_command, media_type, mime_type, source_url o source_message_id, client_id/app, campaign_id, vitesse_msg_id, gsld/wamid, storage_path(raw opcional), event_ts	Envelope ligero con metadata del asset. El binario nunca viaja por Pub/Sub; el Media Handler descarga/normaliza y guarda en /media/...
----------------	---	--

Regla práctica: lo necesario para decidir y enrutar viaja en atributos; lo necesario para reconstruir o reprocesar vive en Storage. En status-events, gsld y status deben propagarse siempre en atributos.

11. Decisiones operativas adicionales

Media handling / archivos multimedia

La arquitectura no debe asumir texto plano únicamente. Cuando un inbound u outbound incluya imagen, PDF, audio, video o documento, el pipeline no transporta binarios por Pub/Sub ni los replica en Firestore. El webhook o processor detecta el asset, emite un media-command y el CRF Media Handler descarga o normaliza el archivo, calcula hash de contenido y lo persiste fuera del event log conversacional.

Regla canónica: el binario se guarda en /media/{app}/{YYYY}/{MM}/{hash}.{ext}. Pub/Sub, Firestore y la proyección viva solo transportan metadata: media_type, mime_type, size, hash, storage_path y, si aplica, signed_url o URL resuelta. Nunca debe viajar el binario ni Base64 pesado en Pub/Sub.

Resultado esperado: el mensaje visible del hilo referencia el asset por storage_path; el histórico completo del asset queda desacoplado del JSON operativo y no presiona el límite de tamaño de Pub/Sub ni el costo de Firestore.

API outbound: respuesta síncrona, envío asíncrono

Cuando un cliente o el frontend invoca POST /send_message (o su equivalente), el API Gateway / CRF Parser genera vitesse_msg_id y local_ref de forma síncrona, persiste el mínimo necesario y devuelve esa identidad en la respuesta HTTP original. Solo después publica outbound-commands y deja que Pub/Sub haga el trabajo pesado.

Contrato recomendado de respuesta: HTTP 202 Accepted + vitesse_msg_id + local_ref + campaign_id + estado inicial=submitted. Así el frontend puede asociar inmediatamente su solicitud a un hilo o mensaje visible y quedarse escuchando en Firestore mientras el envío real avanza por Pub/Sub y Gupshup.

Control de concurrencia y rate limiting hacia Gupshup

Pub/Sub desacopla y absorbe picos, pero no autoriza a bombardear al proveedor. Los consumidores de outbound-commands que escriben a Gupshup deben operar con una cota explícita de concurrencia alineada al TPS contratado por canal o app.

Regla operativa: CRF Sender debe desplegarse con max-instances estricto y concurrency controlado, o bien con suscripción pull / flow control si se requiere un control más fino. Los 429 o señales de rate limit deben disparar backoff, reintento y preservación del mismo vitesse_msg_id/local_ref, nunca un bypass del control de presión.

La unidad de control es el canal/proveedor, no el proyecto completo. Si existen múltiples canales, la plataforma debe poder particionar o serializar por client_id/app para no exceder límites de Gupshup/Meta en un solo canal durante campañas masivas.

Estas tres decisiones completan el contrato operativo del backbone: media desacoplada, API con ID síncrono para el caller y salida outbound bajo control de concurrencia hacia el proveedor.